

}Universidad del Valle de Guatemala
Facultad de Ingeniería
Programación Orientada Objetos
Segundo Ciclo, 2026
Catedrático: Erick Marroquín



Análisis y Diseño

Estudiantes:
Emily Lucila Menchú Menchú - 20472

Fecha: 03/08/2026

Análisis y Diseño

Clases a definir:

- Localidad
- Evento
- Comprador
- Ticket
- VistaEvento
- ControladorEvento
- Main

Relaciones entre clases:

- Evento, Ticket, Comprador y VistaEvento tienen una relación de agregación con la clase ControladorEvento. Estas clases pueden y deben de seguir existiendo aunque el controlador ya no lo haga.
- Localidad tiene una relación de composición con Evento debido a que la localidad no puede existir sin un evento ya que los datos que almacena son específicos de un evento.
- Main tiene una relación de dependencia con el ControladorEvento ya que solo la usa momentáneamente.

Tabla de Propiedades

#	Tipo de dato	Nombre	Visibilidad	Motivos	Clase
1	String	nombre	private	Almacena el nombre de la localidad	Localidad
2	float	precio	private	Para almacenar el precio de los boletos en esa localidad	Localidad
3	int	capacidad	private	Almacena la cantidad de boletos que la localidad puede tener	Localidad
4	int	boletosVendidos	private	Almacena cantidad de boletos comprados	Localidad
5	String	nombre	private	Para que cada evento tenga un identificador	Evento
6	Localidad	localidad1	private	Para guardar la primera localidad	Evento
7	Localidad	localidad5	private	Para guardar la segunda localidad	Evento
8	Localidad	localidad10	private	Para guardar la tercera localidad	Evento
9	String	nombre	private	Guarda el nombre del comprador	Comprador

10	String	correo	private	Para guardar correo	Comprador
11	int	cantidad	private	Para guardar cantidad de boletos que el usuario	Comprador
12	float	presupuesto	private	Para guardar el presupuesto	Comprador
13	int	numero	private	Para guardar el número asignado al ticket	Ticket
14	int	limiteInferior	private	Para guardad el límite inferior	Ticket
15	int	limiteSuperior	private	Para guardad el límite superior	Ticket
16	boolean	valido	private	Para guardar si el ticket es válido	Ticket
17	Scanner	sc	public	Permite leer datos del usuario	VistaEvento
18	Evento	evento	private	Para almacenar un evento	ControladorEvento
19	VistaEvento	vista	private	Para llamar a la vista	ControladorEvento
20	Comprador	comprador	private	Guarda el comprador actual	ControladorEvento
21	Ticket	ticket	private	Guarda el ticket de la solicitud actual	ControladorEvento

Tabla de métodos

#	Tipo de retorno	Nombre	Visibilidad	Parámetro	Motivo	Clase
1	void	main	public	String[] args	Punto de entrada del programa.	Main
2	-	Localidad	public	nombre: String, precio: float, capacidad: int	Método constructor de localidad	Localidad
3	String	getNombre	public	-	Retorna el nombre de la localidad	Localidad

4	float	getPrecio	public	-	Para obtener valor del precio	Localidad
5	int	getBoletosVendidos	public	-	Para obtener cantidad de boletos generales	Localidad
6	int	getBoletosDisponibles	public	-	Obtiene cantidad de boletos disponibles	Localidad
7	void	venderBoletos	public	cantidad:int	Incrementa los boletos vendidos	Localidad
8	-	Evento	public	nombre:String, loc1:Localidad, loc5:Localidad, loc10:Localidad	Método constructor de Evento	Evento
9	String	disponibilidad	public	-	Devuelve disponibilidad total en un String	Evento
10	int	vendidos	public	-	Devuelve total de boletos vendidos	Evento
11	String	disponibilidadPorLocalidad	public	localidad:int	Devuelve disponibilidad de una localidad	Evento
12	String	reporteCaja	public	-	Calcula las ganancias del evento y retorna un String con la información	Evento
13	String	nombreLocalidades	public	-	Devuelve un String de las localidades que existen	Evento
14	String	getNombre	public	-	Devuelve el nombre del	Evento

					evento	
15	Localidad	getLocalidad	public	opcion:int	Devuelve una localidad según opción que se envíe	Evento
16	-	Comprador	public	nombre:String, correo:String	Constructor que asigna todos los atributos	Comprador
17	String	getNombre	public	-	Devuelve el nombre	Comprador
18	String	getCorreo	public	-	Devuelve el correo	Comprador
19	int	getCantidad	public	-	Devuelve la cantidad de boletos solicitada	Comprador
20	float	getPresupuesto	public	-	Devuelve el presupuesto	Comprador
21	void	setCantidadYPresupuesto	public	cantidad:int, presupuesto:float	Asigna los valores de cantidad y presupuesto	Comprador
22	-	Ticket	public	-	Constructor de Ticket	Ticket
23	void	generarNumero	private	-	Genera el número del ticket	Ticket
24	void	generarLimites	public	-	Genera los dos números aleatorios	Ticket
25	void	validarTicket	public	-	Valida si el ticket está dentro del rango	Ticket
26	boolean	getValido	public	-	Obtiene la validez del Ticket	Ticket
27	int	mostrarMenu	public	nombreEvento: String	Muestra el menú y retorna	VistaEvento

					la opción ingresada por el usuario	
28	Comprador	leerComprador	public	-	Pide los datos del comprador	VistaEvento
29	void	mostrarCompra	public	esParcial:boolean localidad:String, cantidad:int, total:float	Muestra mensaje del resultado de la compra	VistaEvento
30	void	mostrarCompraRechazada	public	motivo: int	Muestra mensaje de compra rechazada y motivo	VistaEvento
31	void	mostrarDisponibilidad	public	info:String	Muestra la disponibilidad tanto general como por localidad	VistaEvento
32	void	mostrarReporteCaja	public	info:String	Muestra el dinero generado por la venta de boletos	VistaEvento
33	int	pedirLocalidad	public	localidades:String	Solicita una localidad en base a las opciones existentes	VistaEvento
34	void	mensajeFaltaDatosComprador	public	-	Muestra mensaje de alerta cuando se no hay datos de comprador	VistaEvento
35	void	mensajeOpciónInvalida	public	-	Muestra mensaje de que se ha ingresado una opción inválida	VistaEvento
36	int	leerCantidadBoletosAComprar	public	-	Lee la cantidad de boletos que el usuario quiere	VistaEvento

					comprar	
37	float	leerPresupuesto	public	-	Lee el presupuesto del comprador	VistaEvento
38	-	ControladorEvento	public	-	Constructor del controlador	ControladorEvento
39	void	iniciar	public	-	Método para inicializar el programa de compra de tickets y muestra el menu	ControladorEvento
40	void	nuevoComprador	private	-	Crea un nuevo comprador y asigna la nueva instancia	ControladorEvento
41	void	solicitudCompra	private	-	Maneja el proceso de compra y validación de boletos	ControladorEvento
42	void	disponibilidad	private	-	Gestiona la obtención y muestra de disponibilidad total	ControladorEvento
43	void	disponibilidadPorLocalidad	private	-	Gestiona la obtención de disponibilidad de una Localidad para enviársela a la vista	ControladorEvento
44	void	reporteCaja	private	-	Muestra el dinero generado	ControladorEvento

6. ¿Cómo clasifica cada una de sus clases según el patrón MVC (Model, View, Controller)? Justifique brevemente el rol que cumple cada una dentro de esa clasificación.

Model:

- Localidad

Representa una localidad en el evento y guarda información relacionada a la localidad. Además, realiza operaciones y cálculos sobre sus datos.

- Evento
Representa el evento y contiene tres localidades. También se encarga de hacer cálculos u operar sobre sus propios datos.
- Ticket
Representa el ticket de compra. Realiza varios cálculos que tienen que ver con la lógica de negocio.
- Comprador
Representa al comprador y almacena información que luego será necesaria para hacer la solicitud de compra.

View:

- VistaEvento
Es la interfaz que se comunica directamente con el usuario. Le solicita y muestra información al usuario.

Controller:

- ControladorEvento
Controla y coordina la comunicación entre la vista y modelo. Recibe las acciones o información ingresada por el usuario, utiliza a las clases del modelo para ejecutar lógica del programa y envía resultados a la vista para que se los muestre al usuario.

Clase extra:

- Main
Esta clase no conforma el patrón MVC pero es el punto de entrada del programa lo que ayuda a que el programa se ejecute.

7. ¿Qué ventaja ofrece separar la lógica de negocio (Model) de la interacción con el usuario (View/Controller) en este sistema? ¿Qué problema evita esta separación si en el futuro se necesita modificar el algoritmo de asignación de tickets?

La principal ventaja es que cada clase tiene una responsabilidad específica. En el Model se almacena información y se realizan cálculos y validaciones; View tiene la única responsabilidad de interactuar con usuarios, pidiendo o mostrando información; y el Controller gestiona la comunicación entre Model y View. Otra ventaja es la organización del código ya que de esa manera es más fácil entender y mantener el código. Además, esta separación ayuda a proteger la integridad de los datos, ya que el usuario no interactúa directamente con las clases del modelo, sino que a través del controlador, de esta manera se evitan modificaciones incorrectas sobre datos y lógica del sistema.

El problema que se evita si en un futuro el algoritmo de asignación de tickets cambia es que las modificaciones se limitarán a las clases responsables de esa lógica, como Ticket y, si fuera necesario, ControladorEvento. Clases como VistaEvento no tendrían que modificarse porque no maneja nada de esa lógica y así la forma en la que el usuario interactúa con el sistema seguiría siendo la misma. Al tener las responsabilidades bien separadas y aisladas, los cambios en una funcionalidad tienen menos probabilidad de afectar a otras partes del programa.

Diagrama UML

